Document Processing and Retrieval: A Practical Guide

Page 1: Overview

This guide explains a simple document retrieval system. The system accepts PDF, Word, and text files, extracts their content, divides the content into smaller passages, and stores those passages in a vector database.

The goal of retrieval is to find passages that are useful for answering a question. A retriever does not write the final answer. Its responsibility is to locate relevant evidence and return that evidence with useful metadata.

A typical pipeline has five stages: document loading, text cleaning, chunking, embedding generation, and vector storage. Each stage prepares information for the next stage.

Document Processing and Retrieval: A Practical Guide

Page 2: Document Loading and Metadata

A document loader reads a file and converts its contents into Document objects. In LangChain, a Document usually contains page_content and metadata. The page_content field stores extracted text.

Metadata describes where the text came from. Useful fields include the original filename, page number, file type, and an optional document identifier. Metadata makes it possible to filter results and cite their source.

PDF loaders often return one Document per page. Word and plain-text loaders may return a different number of Document objects depending on the loader. A pipeline should handle missing page numbers rather than assuming every file has pages.

Document Processing and Retrieval: A Practical Guide

Page 3: Cleaning and Chunking

Extracted text can contain repeated spaces, unnecessary blank lines, headers, or formatting artifacts. Text cleaning should remove unnecessary whitespace while preserving meaningful paragraph boundaries and punctuation.

Chunking divides long text into smaller passages. A chunk should contain enough information to be meaningful, but it should not be so large that retrieval returns too much unrelated content.

Chunk size and overlap are configuration choices. For example, a system might use chunks of about 800 characters with an overlap of 100 characters. Overlap helps preserve context when an important sentence falls near a chunk boundary.

Document Processing and Retrieval: A Practical Guide

Page 4: Embeddings and Vector Databases

An embedding model converts text into a numerical vector. Texts with related meanings may have vectors that are close to each other in the embedding space. This allows semantic search to find relevant passages even when the query does not use the exact same words.

A vector database stores vectors together with their text and metadata. ChromaDB is one example. A collection groups related records, and each record should have a stable unique identifier.

When documents are ingested more than once, a system should avoid creating duplicate records. Stable IDs and update operations can help keep the collection consistent. A persistent database keeps data available between application runs.

Document Processing and Retrieval: A Practical Guide

Page 5: Search and Ranking

Semantic search compares the query embedding with stored chunk embeddings. It is useful for questions expressed in different words from the source material.

Keyword search looks for exact terms and important words. BM25 is a common ranking method for keyword retrieval. Combining semantic and keyword search can improve coverage because the two methods find different kinds of matches.

A reranker takes an initial set of candidate passages and scores them again using both the question and each passage. The highest-ranked passages can then be selected as evidence for the next stage.

Document Processing and Retrieval: A Practical Guide

Page 6: Evaluation and Good Practices

A retrieval system should be tested with questions whose answers are located on different pages. Tests should check not only whether relevant text is returned, but also whether the filename and page metadata are correct.

Useful test questions for this guide include: What information is stored in metadata? Why does chunk overlap help? What is the role of an embedding model? How can duplicate records be prevented? How does a reranker work?

A reliable pipeline should report unsupported file types clearly, handle empty documents, preserve source metadata, and make database behavior predictable. Small tests after each stage make problems easier to identify.